

Music Capstone Technical Report

Maxwell Elim, Maximilian Van Zandt, Haoting Yuan, Grant Zou

2025-12-9

Contents

1 Introduction and Research Question	2
2 Challenges and Alternative Proposals	2
3 Data Collection	2
3.1 Data Sources	2
3.2 Dataset Structure	3
4 Feature Engineering	3
4.1 Feature Groups	3
4.2 Feature Extraction Pipeline	5
5 Unsupervised Learning: Feature Group Clustering	5
5.1 General Clustering Methodology	5
5.2 Edge Features Clustering	6
5.3 Texture Features Clustering	8
5.4 RGB Features Clustering	10
5.5 HSV Features Clustering	12
6 Supervised Learning: Predicting Album Success from Cover Art	15
6.1 Labeled data construction	15
6.2 Visual feature set and link to the clustering stage	16
6.3 Train-test split and evaluation framework	17
6.4 Model specifications	17
6.5 Predictive performance	18
6.6 Feature importance, discussion, and limitations	20
7 Assumptions and Limitations	21
Appendix A: Data Dictionary	23
Appendix B: Code	24

1 Introduction and Research Question

Due to the advent of new internet technologies, online streaming has become the most popular way for people to listen to the music that they want. Companies such as Spotify are worth billions of dollars and are projected to continue to grow. Alongside them are music artists and their producers and record labels who still aim to create the most popular songs. While much analysis has been done on the audio characteristics of hit songs, an area which remains under-researched is how album cover art impacts the performance of songs. On music streaming platforms, album cover art acts as a visual placeholder for a song and is prominently featured when a song is playing. Cover art can act as a user’s first impression of an album and influence users to listen by capturing their attention for example. As such, this study investigates whether one can predict whether an album will join the Billboard 200 based on visual characteristics of its cover art.

2 Challenges and Alternative Proposals

This study considered several other ways to measure the importance of cover art but decided that they were not worth pursuing. They are outlined in this section. Initially, this study was aimed at using album cover art to predict the performance of a song in the Billboard Hot 100. However, this ran into a problem: all the songs in an album share the same cover art but can have drastically different performances. If multiple songs from the same album made the Hot 100, this would result in problems for our model which wouldn’t be able to use the album cover art to explain the differences in performance. Additionally, if this study were to try and predict whether a song made the Hot 100 or not, sampling multiple songs from the same album that made or didn’t make the Hot 100 would result in the same problem. Initially, this study considered predicting song performance based on audio features collected from the Spotify API. Unfortunately, Spotify deprecated their audio features API endpoint, making it impossible to get audio features that way. This study investigated other sources of audio features, but none were as simple, comprehensive, or free like Spotify’s. For these reasons, this study moved to investigating cover art. Another intuitive idea is to incorporate audio features into our analysis and find patterns with visual features. However, finding album-wide audio features poses many challenges. First, albums often don’t have consistent audio features. Individual songs in an album can vary dramatically, and averaging measurements like beats per minute can lose information. Second is that given the difficulty in defining album audio features, there are very few data sources that provide album level audio data, making it impractical to pursue. We searched for album level genre data and found [getgenre.com](#). However, approximately 25% of albums from the Billboard 200 do not have a genre associated with them, so we abandoned this approach.

3 Data Collection

3.1 Data Sources

We combined data from two primary sources:

1. Album Cover Images and Metadata (MusicBrainz API)

We used the MusicBrainz API to access a comprehensive database of album releases. This database includes all albums, not just those that appeared on the Billboard 200. This allowed us to construct a dataset with both positive cases (albums that appeared on the Billboard

200) and negative cases (albums that never made the Billboard 200). For each album, we obtained a link to a cover image, and album metadata including Artist, Country, Date, and Label Names. We used Python code to visit the link to the cover image and download it as a jpg file. We collected albums released in a 28-week window from January 2025 to July 2025. Due to limitations of the MusicBrainz API, some requests got dropped and we did not collect the data for all albums in the time frame. Any album request to the MusicBrainz API that failed was dropped from the data. Additionally, the MusicBrainz API did not have the cover art of every album. If an album is missing its cover art, it is dropped from the dataset.

2. Billboard 200 Chart Data

This is the Weekly Billboard 200 chart positions. These are the positive cases that performed well. Billboard information is available from the Python package `billboard.py`. Data we collected from the Billboard included the rank of an album, its peak position, last week's position, the number of weeks that it has been present on the chart, and whether it is new to the chart. We collected chart data from July 2025 to September 2025. Every album that was collected from the Billboard chart was also collected from the MusicBrainz API to get other metadata.

3.2 Dataset Structure

Each observation in our dataset represents a single album and contains:

- Album cover image (resized to 180 x 180 pixels for consistency)
- Album metadata (title, artist, release date, record label)
- Response variable: Indicator of Billboard 200 appearance.

4 Feature Engineering

This section outlines how we took the data described above and extracted visual features from album covers for analysis and model-creation. For image processing, this study employs the `scikit-image` library in Python, which extracted and computed 70 features. In Section 5, we outline the methodology behind preserving the feature groups throughout the analysis phase. But for now, this section explains how these features are computed and how they reveal important nuances within the album cover images.

4.1 Feature Groups

Beginning with the structure and the overall intensity of the image, the **edge feature group** contains 2 features that capture these characteristics. Visually, edge features can be thought of as the amount of contrast in an image or the amount of sharp edges, shadows, etc. The way edge features measure structural complexity is through gradients. In short, we have two gradient kernels that calculate the gradients (black/white intensity) in the x and y direction (images are 2D so no

need for anything more). This technique is known as the sobel operator.

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} * I, \quad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix} * I$$

where I is the grayscale image and $*$ denotes convolution.

At each pixel, we calculate the relative distance (magnitude) of that pixel in relation to the gradient $G = \sqrt{G_x^2 + G_y^2}$, which is just the simple magnitude formula from linear algebra but adapted here. Higher values of this indicate sharper edges and lower values indicate softer edges. In Python, the sci-kit image library (and the filters module) has a function called `skimage.filters.sobel` which extracts two key features from this field: (i) **sobel_mean** which is just the mean of the gradient magnitude we calculated earlier but across all the pixels and (ii) **edge_density** which is the proportion of G above the 90th percentile (proportion of “sharp” edges). Overall, higher values of **sobel_mean** and **edge_density** indicate more complexity while lower values indicate less complexity (minimalist designs).

Beyond the contrast and brightness of an image, it is often helpful to examine the innate patterns within the image. We often describe these features as the **texture feature group** because in images, there are often elements that we relate to via touch and feel, even in a 2D setting. To understand the innate patterns in an image, we start by evaluating a single pixel’s brightness in relation to 8 of their neighbor pixels (evaluating in grayscale) with the following formula:

$$\text{LBP}(x_c, y_c) = \sum_{p=0}^7 s(g_p - g_c) \cdot 2^p$$

where g_c is the center pixel intensity, g_p are the 8 neighbor intensities, and

$$s(x) = \begin{cases} 1 & \text{if } x \geq 0 \\ 0 & \text{if } x < 0 \end{cases}$$

Then, we assign a number to this pixel based on the relationships we found; this number is an 8 digit-long binary (0-256) with each digit representing a relationship with one neighboring pixel. Mathematically, this is called Local Binary Patterns (LBP). Finally, we repeat this process with the other pixels in the image and bin up the resulting LBP numbers (256) into a 32-bin histogram (dividing 256/8 to reduce the dimensionality). Typically, higher LBP-numbered bins indicate sharper edges and more complex patterns.

Now that texture and edge features have been extracted, it is important to examine the two color-based feature groups of an image: **Hue-Saturation-Value (HSV) feature group** and the **Red, Green, and Blue (RGB) feature group**. RGB is perhaps the most obvious feature group out of all of them as it quantifies the presence of different colors and color-combinations. For feature extraction, RGB features consist of the 4 statistical moments (mean, standard deviation, skewness, and kurtosis) calculated at the pixel level (normalized) for each of the 3 color channels (red, green, blue), making 12 features in total. For the HSV group, this can be thought of as the saturation or mood of an image. Hue represents the color on the color wheel; saturation measures the color intensity so how close or far it is from grayscale; and value captures brightness from a fully black image to a white, illuminated image. For example, a picture of fire may have a high Hue for warmer, orange/yellow colors, a high Saturation for the vivid colors, and medium Value to capture

natural brightness without appearing too neon or unrealistic. To extract HSV features, we use the following function from sci-kit image:

```
hsv = color.rgb2hsv(img)
hist_h = np.histogram(hsv[...,0], bins=8, range=(0,1), density=True)[0]
hist_s = np.histogram(hsv[...,1], bins=8, range=(0,1), density=True)[0]
hist_v = np.histogram(hsv[...,2], bins=8, range=(0,1), density=True)[0]
```

Each channel (H, S, and V) is given 8 bins with pixels sorted into those bins based on their values for H, S, and V. 8 bins for each of 3 channels yields 24 total features.

4.2 Feature Extraction Pipeline

In order to load the album cover images and using sci-kit image to extract features, we followed a 5-step approach. **First**, we loaded the images (as RGB images) and standardized the sizing to 180x180 pixels. **Second**, we apply the luminosity formula $I_{\text{gray}} = 0.2125R + 0.7154G + 0.0721B$ to change the images to grayscale. **Third**, we convert the RGB images to the HSV setting as described in Section 4.1. **Fourth**, we compute the 24 HSV features, 12 RGB features, 32 LBP (texture) features, and 2 Sobel (edge) features. **Fifth** and finally, we concatenate the features into a single 70-dimensional feature vector for each album, where the distance between points represents similarity in visual features.

5 Unsupervised Learning: Feature Group Clustering

In order to make informed decisions surrounding the data labels and have productive supervised learning, this study begins with an unsupervised learning phase. This phase seeks to cluster album cover features to discover what features are most important and how each feature is related to the others. This, in turn, will inform decision-making surrounding the features used in supervised learning models. Initially, this study sought to cluster all 70 features in one space. However, it quickly became clear that this mass-clustering effort would obfuscate the true relationships among features. Moreover, the distance metrics and clustering methods may need to differ among feature groups. Thus, this study splits up the 70 features into their feature groups (HSV, RGB, Edge, Texture) before clustering. This way, the clustering methods can be specialized to the feature groups with an understanding of what they represent mathematically.

5.1 General Clustering Methodology

For each feature group, this study followed the same clustering methodology. This systematic approach ensures that similar reasoning can be applied when evaluating feature importance and that cross-feature-group analysis is more robust and efficient.

The clustering follows a 5-step approach which is in line with typical clustering methodology. The **first** step is preprocessing which standardizes the measurement units in each feature for effective distance calculations. For low-dimensional features, it's appropriate to use standard z-score normalization (StandardScaler) whereas for high-dimensional features, this study went further and applied PCA for dimensionality reduction following the z-score normalization. The rationale behind applying PCA for high-dimensional features is that these high-dimensional spaces contain

substantial noise. This noise obscures the clustering structure and increases computational complexity because the data points become further out. This phenomenon is dubbed “the curse of dimensionality” and renders distance-based clustering methods less effective. Thus, PCA is the standard practice in these scenarios for reducing noise with the trade-off of potential information loss which will be discussed in Section 7 of this report. The **second** step is the clustering step which, for this study, involves applying K-means with Euclidean distance. The rationale behind K-means is that hard cluster assignments are important along with interpretable results. While Gaussian Mixture Models (GMM) might capture uncertainty and have a slightly higher accuracy in cluster assignments, the soft assignments are not desirable given that the goal of this unsupervised stage was to inform label-creation for the supervised phase. Moreover, GMM assumes distributional characteristics in the data that we had not verified. Given that the number of albums is 315, we tested $K \in \{2, 3, \dots, 11\}$ clusters. An upper bound of 11 was chosen to balance thorough exploration of cluster memberships along with interpretability. K-means was run with 20 random initializations (nstart=20) to ensure convergence to global optima. The **third** step involves choosing the optimal K which was mainly based on the Silhouette score, defined as $s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$ where $a(i)$ is the mean within-cluster distance and $b(i)$ is the mean between-cluster distance for point i , the Calinski-Harabasz index defined as $CH = \frac{SS_B / (K-1)}{SS_W / (n-K)}$ where SS_B is the between-cluster sum of squares, SS_W is the within-cluster sum of squares, n is the number of observations, and K is the number of clusters, and the Davies-Bouldin index defined as $DB = \frac{1}{K} \sum_{i=1}^K \max_{j \neq i} \left(\frac{s_i + s_j}{d_{ij}} \right)$ where s_i is the average distance of points in cluster i to the cluster centroid and d_{ij} is the distance between cluster centroids i and j . The selection of K was based on the highest Silhouette score while also considering interpretability. The **fourth** step is the stability assessment, involving an assessment of the Adjusted Rand Index (ARI) across multiple random seeds. The ARI measures the agreement across clusters and their true labels. An ARI of 1 indicates perfect stability and an ARI of 0 indicates severe instability. This ensures that our results are not initialization-dependent. The **fifth** and final step of this clustering phase is interpretation based on 2D projection visualizations to help assign qualitative labels to clusters. These cluster groups were saved into separate folders in the data repo for further inspection.

5.2 Edge Features Clustering

The edge feature group only includes 2 features: sobel mean and edge density with high values of both indicating high visual complexity and lower values indicating simpler designs. Since this is only a 2-dimensional feature set, the clustering needs only z-score normalization, no PCA. As discussed in Section 5.1, K-means with Euclidean distance was performed. The optimal K was determined to be $K = 2$ as it demonstrated the highest Silhouette score of 0.525, lowest Davies-Bouldin index at 0.653 and an ARI of 1, indicating perfect stability across all random seeds. These results indicate a natural binary separation between the two edge features. Breaking this down, the first cluster (Cluster C0) can be interpreted as more complex covers, capturing 46.3% of the variation and $n = 146$ covers. This cluster had a sobel mean of 0.076 and a mean edge density of 0.127 which are both above the median. Visually, the album covers in this cluster can be described as being more visually dense (more patterns) and contained higher information.

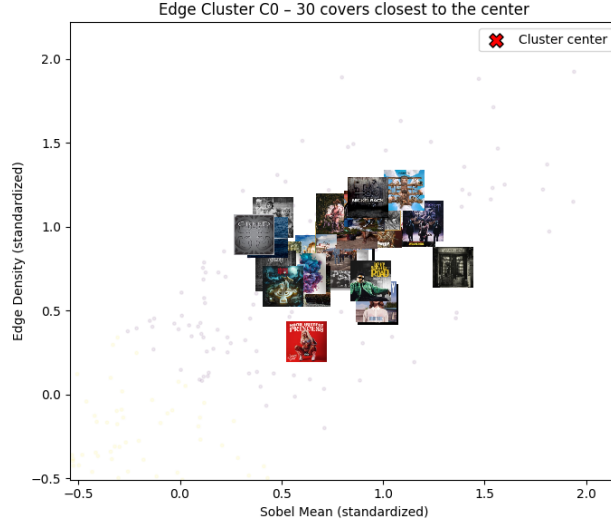


Figure 1: Cluster C0 for Edge Feature Group

The second cluster (Cluster C1) was far simpler in visual nature. This cluster captured 53.7% of the variation and $n = 169$ covers. The mean sobel was 0.034 and the mean edge density was 0.057 which are both below the median. Visually, these album covers can be described as more minimalistic, with more solid colors, uncluttered layouts, and simpler typography.

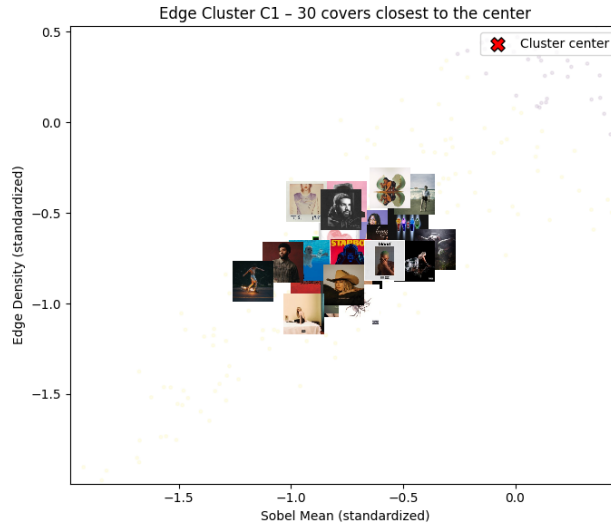


Figure 2: Cluster C1 for Edge Feature Group

Overall, the edge feature clustering yielded well-separated, stable clusters, evidenced by high Silhouette scores and an ARI of 1. This binary structure is analytically fascinating, as it indicates a clear difference between more minimal and complex visual styles in album cover art. This will prove useful in the supervised learning phase.

5.3 Texture Features Clustering

The texture feature group is significantly more high-dimensional than the edge feature group with 32 features. Thus, PCA was applied after z-score normalization (via StandardScalar) to reduce the dimensionality of the data (discussed in Section 5.1). As shown in the figure below, the elbow of the scree plot occurs at around PC2-PC4. Given the unclear nature of the scree plot, K-means was run on 2 PCs, 3 PCs, and 4 PCs with the Silhouette scores being the deciding factor.

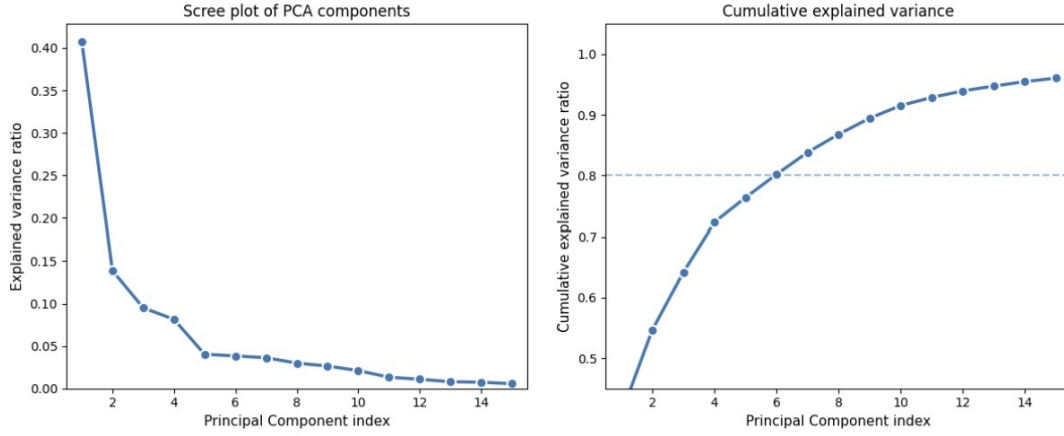


Figure 3: Scree Plot for Texture Feature Group and Cumulative Explained Variance Plot for the Texture Feature Group

After running the K-means clustering on 2, 3, and 4 PCs, the Silhouette scores reveal that 2 PCs is most appropriate (Figure 5). Past 2 PCs, the Silhouette scores actually fall below 0.5, indicating poor separability. Additionally, the feature loadings past PC2 become unstable, suggesting that only noise is captured past PC2.

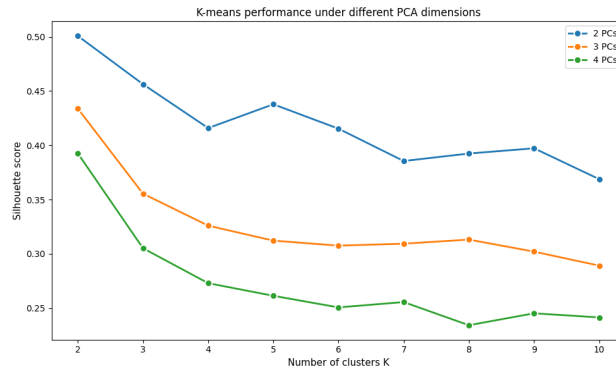


Figure 4: Testing the K-Means Performance via Silhouette Score Under Different PCs for Texture Feature Group

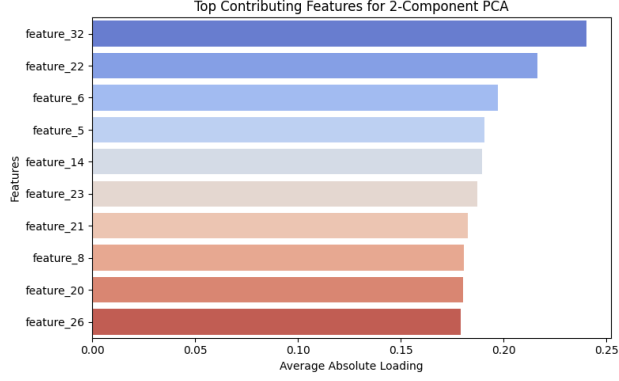


Figure 5: Examining the Texture Group Feature Loadings (contributions to explained variance) in the 2-PC Setting

Overall, the results of PCA were that 2 PCs capture around 54.66% of the variance with PC1 capturing 40.78% and PC2 capturing 13.88%. While not all the variance was explained, as discussed above, the cluster separability becomes poor past 2 PCs. Moreover, 10 PCs would be needed to explain $\sim 90\%$ of the variance, making interpretability more difficult. In turn, these results suggest that much of the texture feature group is noise and that 2 PCs are sufficient for capturing the true cluster separability. As for the K-means results, the Silhouette score was 0.501 with an optimal $K = 2$ (lowest Davies-Bouldin index of 0.855). This indicates good separability between clusters. Furthermore, the clustering is stable across different random seeds, evidenced by an ARI of 1. Breaking the clustering down further, the first cluster (Cluster C0), captures around 31.4% of the variation with $n = 99$ album covers. The album covers in this cluster can be described as smoother and simpler in terms of texture patterns with lower PC1 values.

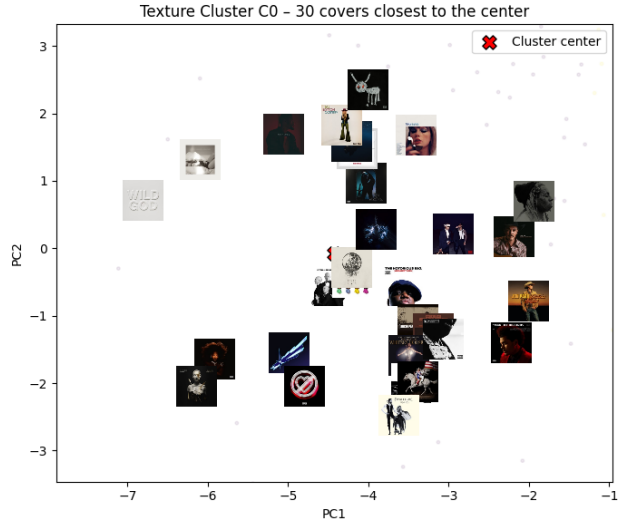


Figure 6: Cluster C0 for Texture Feature Group

The second cluster (Cluster C1) captured a greater percentage of the variation ($\sim 68.6\%$) with $n = 216$ album covers. The album covers in this cluster had more complex and varied textures along with more intricate patterns. Moreover, there were higher PC1 values in this cluster.

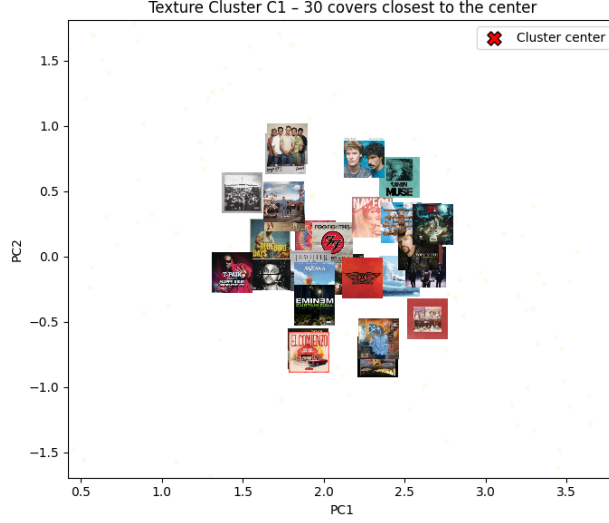


Figure 7: Cluster C1 for Texture Feature Group

Overall, the texture feature group clustering showed moderate discriminative power with a Silhouette score of 0.501 and reasonably balanced clusters. These results were an improvement over the full 32-dimensional feature space in the group, providing evidence that PCA was the right choice in this situation. Even with 45% of the variation left unexplained, the clustering separability rapidly deteriorates past 3 PCs, so this clustering on 2 PCs strikes the right balance between explainability and cluster separability for this study.

5.4 RGB Features Clustering

This subsection describes the RGB-based clustering pipeline and reports the diagnostics we used to choose both the PCA representation and the number of clusters, as well as a detailed analysis of feature contributions to the principal components.

We start from a set of summary RGB features computed for each cover: the mean, standard deviation, skewness and kurtosis of the R, G and B channels (12 features in total). Before PCA, all features are standardized per the methodology in Section 5.1. A full PCA on the standardized RGB matrix reveals a strongly concentrated variance structure. The scree plot shows that PC1 explains 47.3% of the total variance and PC2 a further 23.3%, while PC3 and PC4 explain only 12.3% and 5.3% respectively; later components each contribute less than 5%. The cumulative explained-variance curve crosses 80% between the third and fourth components and then flattens, indicating a clear elbow around 3–4 dimensions. This suggests that most of the meaningful RGB variation can be captured in a very low-dimensional space.

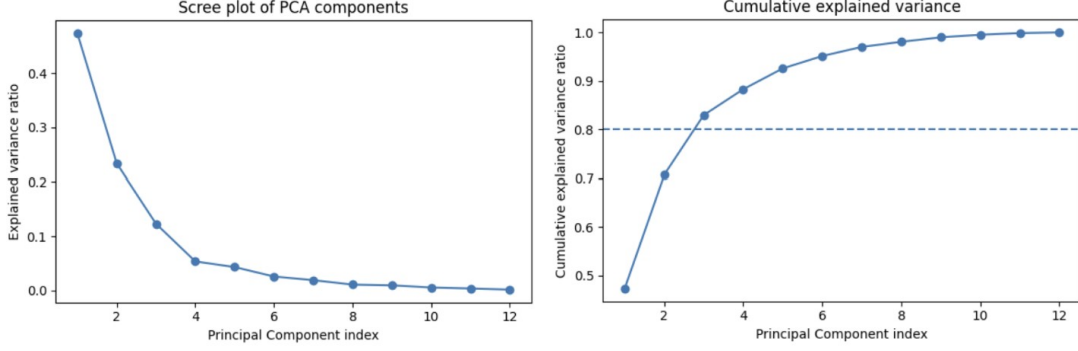


Figure 8: Scree Plot for RGB Feature Group and Cumulative Explained Variance Plot for the RGB Feature Group

We considered embeddings with 2, 3, and 4 components. For brevity, the full 3 matrices for the Calinski–Harabasz and Davies–Bouldin indices as well as the Silhouette scores are included in the appendix.

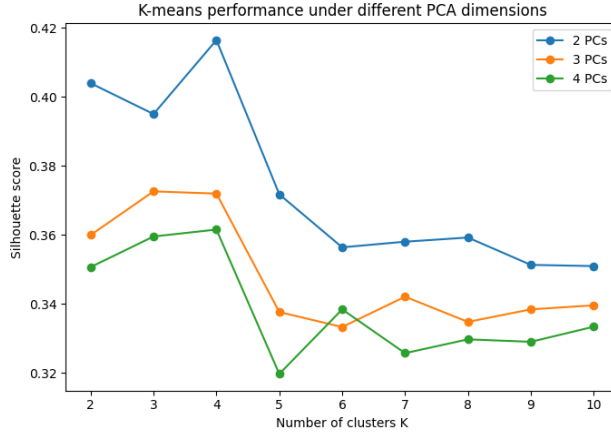


Figure 9: Testing the K-Means Performance via Silhouette Score Under Different PCs for RGB Feature Group

The Silhouette score peaked at **$K = 4$ with a value of 0.416**, higher than all other K and higher than the corresponding maxima for the 3D and 4D embeddings. The Calinski–Harabasz index increases monotonically with K (as it often does), so it mainly indicates that solutions with more clusters are more compact in absolute terms rather than pointing to a single clear optimum. The Davies–Bouldin index is lowest at $K = 5$ (0.739) and close to that at $K = 4$ (0.766). Taken together, these metrics suggest that $K = 4$ offers a good compromise between cluster separation, compactness, and interpretability in the 2D PCA space. We also assessed the stability of the $K = 4$ solution on PCA(2) using the Adjusted Rand Index (ARI) across multiple random seeds. The clustering is highly stable across different random seeds, evidenced by a mean ARI of 0.985. Feature loadings reveal that for PC1, contributions aggregated by statistic type are approximately: skewness 34.7%, means 31.6%, standard deviations 18.1%, and kurtosis 15.6%. Aggregated by channel, **PC1** contributions are B: 38.1%, G: 38.5%, R: 23.5%. Thus PC1 captures a combination of overall RGB

channel balance and asymmetry, with slightly more weight on the green and blue channels; covers with high PC1 scores tend to have strong channel skewness and higher average intensities. For **PC2**, the picture is different: contributions by statistic type are dominated by standard deviations (34.1%) and kurtosis (30.1%), with smaller roles for means (22.9%) and skewness (12.9%). By channel, PC2 is heavily driven by the red channel (46.8%), followed by green (30.1%) and blue (23.0%). This suggests that PC2 primarily encodes variability and heavy-tailed behavior, especially in the red channel - distinguishing covers with intense red contrasts and extreme highlights/shadows from more uniformly colored ones. When we extend the analysis to four PCs, we find that **PC3** and **PC4** are dominated by standard deviation and kurtosis terms (together about 85% of PC3 and over 80% of PC4), with PC3 focusing on G/B variability and PC4 almost entirely on red-channel kurtosis. These components capture more specialized, tail-driven patterns rather than broad color trends. Because our clustering goal is to uncover a small number of interpretable RGB styles rather than rare extremes, we retained only the first two PCs for k-means and treat PC3–PC4 as secondary.

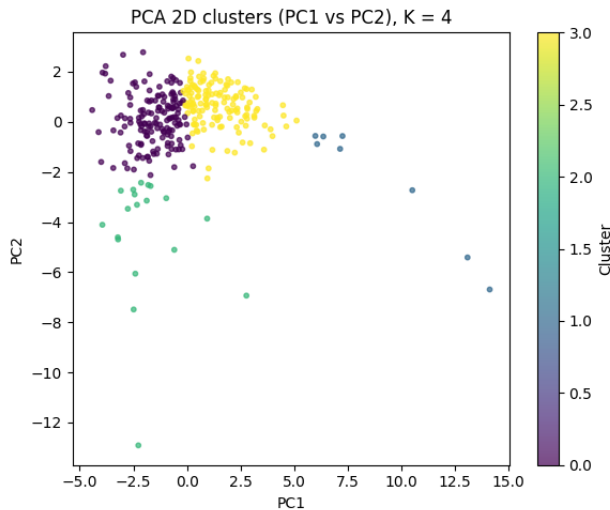


Figure 10: Cluster Assignments for RGB Feature Group Comparing PC1 with PC2

Overall, this analysis shows that the chosen $\text{PCA}(2) + K = 4$ clustering is both statistically well-supported and interpretable: PC1-PC2 summarize around 70% of RGB variance, yield the strongest and most stable cluster structure, and can be linked back to meaningful concepts such as overall brightness, color balance, and channel-specific contrast. These insights also highlight which RGB summary features are most important and which can be de-emphasized in later stages of feature engineering.

5.5 HSV Features Clustering

This subsection documents the HSV-based clustering pipeline and mirrors the analysis carried out for the RGB summary features. Here we work with 24 standardized HSV statistics per cover (e.g. means and dispersion measures for hue, saturation, and value). Features are preprocessed as described in Section 5.1 before PCA. We first fit a full PCA model and examined the explained-variance profile. The scree plot shows a much more gradual decay than in the RGB case: PC1 explains 13.8% of the total variance, PC2 explains 10.6%, PC3 9.2%, PC4 7.8%, and PCs 5–10 each

contribute between roughly 4% and 6%. Cumulatively, the first 10 components explained about 73% of the variance, and we need around 12 components to pass the 80% threshold indicated by the dashed line in the cumulative variance plot. This pattern suggests that HSV information is spread over many directions: instead of one or two dominant axes, we have a set of moderately important components capturing different aspects of hue, saturation, and brightness.

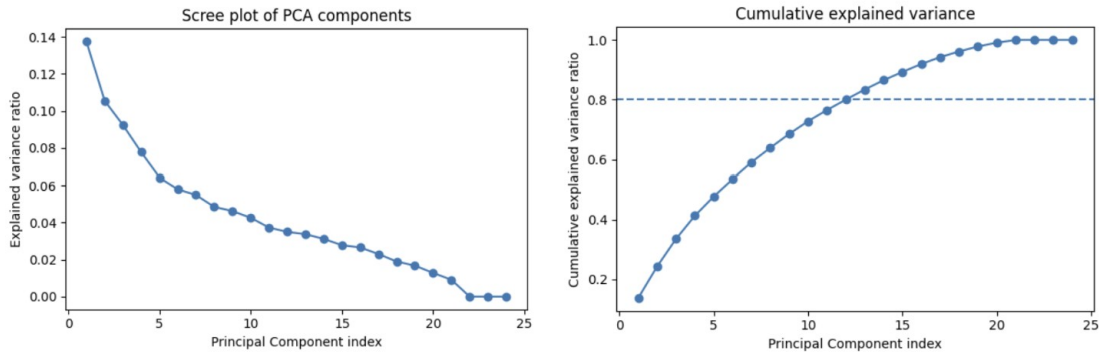


Figure 11: Scree Plot for HSV Feature Group and Cumulative Explained Variance Plot for the HSV Feature Group

Despite this more diffuse structure, we still needed a low-dimensional embedding for visualization and k-means. We considered PCA embeddings with 2, 5, and 8 components, running k-means with $K \in \{2, \dots, 10\}$. The “K-means performance under different PCA dimensions” plot shows that the 2D PCA representation again yields the strongest cluster structure: its Silhouette peaks at 0.391 for $K = 3$, while the best scores for 5 PCs and 8 PCs are much lower (0.204 at $K = 4$ and 0.162 at $K = 10$, respectively). This indicates that adding extra HSV components mostly introduces noise for k-means rather than revealing clearer clusters. Therefore, we focused on PCA with two components when deciding on K and when visualizing clusters.

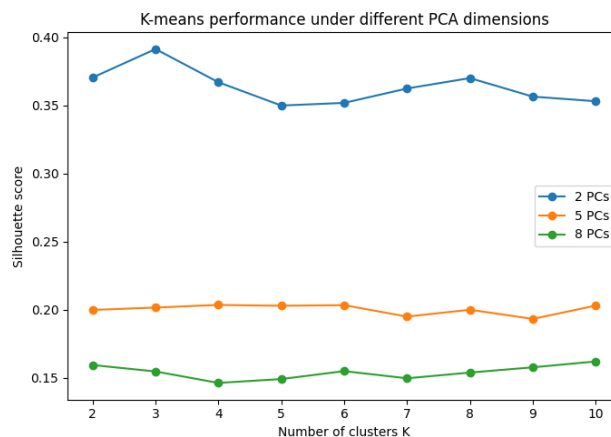


Figure 12: Testing the K-Means Performance via Silhouette Score Under Different PCs for HSV Feature Group

To further diagnose the choice of K on the 2D HSV embedding, we computed three internal valida-

tion metrics for K from 2 to 11. Again, for brevity, the full matrices are included in the appendix. The Silhouette score clearly favors $K = 3$, which achieves the global maximum 0.391. The Calinski–Harabasz index is also high for $K = 3$, and although it attained a slightly larger value at $K = 8$, the gain is modest and comes at the cost of a much more complicated partition. The Davies–Bouldin index decreases as we moved from $K = 2$ to $K = 3$ (1.075 to 0.865), and remains in a similar range for 3-8 clusters (255.1 to 256.1). Given this balance of separation, compactness, and interpretability, we adopted $K = 3$ on the 2D HSV embedding as our main configuration. The “PCA 2D clusters (PC1 vs PC2), $K = 3$ ” plot confirms that the three clusters occupy distinct regions in the PC1–PC2 plane, albeit with more overlap than in the RGB case, which is consistent with the more diffuse eigenvalue spectrum.

We also assessed clustering stability via the Adjusted Rand Index (ARI) across multiple random initialization for $K = 3$ on PCA(2). The results show the clustering is perfectly stable across different random seeds, evidenced by an ARI of 1. Finally, we examined the feature-contribution tables for the first two HSV components in more detail. For **PC1**, the largest contributions come from value-related statistics, followed by saturation, with hue features contributing only modestly. In practical terms, most of the top-ranked features for PC1 are measures of the *overall brightness level* and its spread across the image (e.g. mean and standard deviation of V), together with coarse measures of how saturated the cover is on average. Hue appears in PC1 mainly through weaker higher-order terms, indicating that PC1 is not about which particular colors are present, but about how light/dark and washed-out/vivid the cover is. This motivates interpreting PC1 as a *global brightness-intensity axis*: low PC1 scores correspond to dark, low-saturation covers, whereas high PC1 scores capture bright, high-impact covers with strong value contrast. **PC2** shows a different pattern. Here, the dominant contributions shift toward *hue dispersion and saturation variability*, with value playing a secondary role. Features that measure how concentrated or spread-out the hue distribution is (e.g. hue variance or higher-order dispersion statistics) appear among the top contributors, as do statistics capturing how much saturation fluctuates across the cover. Intuitively, PC2 distinguishes covers that use a relatively uniform color palette (tight hue distribution, mild saturation variation) from covers that mix many different hues and saturation levels within the same artwork. Because PC2 is orthogonal to PC1, it reorders covers primarily by *color diversity and contrast* rather than by overall brightness. This feature-contribution analysis has two direct implications for later modeling: **First**, it highlights a small set of HSV features - especially value mean/dispersion and hue/saturation variability - that drive the dominant axes of variation and therefore deserve priority in downstream supervised models or in any handcrafted feature refinements (for example, constructing interactions between value contrast and hue diversity). **Second**, features that consistently receive negligible contributions to both PC1 and PC2 and only become important in much later components are likely to encode very fine-grained or rare patterns. These can be safely down-weighted, combined, or even dropped altogether.

In summary, although HSV variance is spread over many components, the first two principal components still provide a useful and interpretable low-dimensional representation. Together they explain about 24% of the total variance, support a well-behaved and extremely stable three-cluster k-means solution, and can be understood in terms of meaningful HSV properties: a primary axis of brightness and overall intensity (PC1) and a secondary axis of color diversity and saturation contrast (PC2).

6 Supervised Learning: Predicting Album Success from Cover Art

The first stage of the project used unsupervised methods to understand the structure of visual features extracted from album covers. Building on these exploratory findings, the second stage constructed supervised models that predict whether a newly released album will enter the Billboard 200 chart within a fixed time window after release, using cover art features and basic metadata as predictors. This section describes how we constructed the labeled data set, defined the visual feature set, specified the train–test split and evaluation framework, and estimated and interpreted our supervised models.

6.1 Labeled data construction

6.1.1 Successful albums: Billboard 200 We defined “successful” albums using the Billboard 200 chart. Using the `billboard` Python package, we collect all albums that appear on any Billboard 200 chart between 1 January 2025 and 1 July 2025 (inclusive). For each chart, we extract the album’s title and artist, and then deduplicate across weeks using a unique album identifier (e.g. a title–artist pair or release identifier), so that an album that appears on the chart in multiple weeks is counted only once.

After deduplication and cleaning, this yielded a set of 461 unique “successful” albums. These are labeled with

- `label = 1`.

6.1.2 Normal albums: MusicBrainz sample To obtain a control group, we sample albums from MusicBrainz. We start from the table `musicbrainz_albums_2024-12-01_to_2025-07-01`, which contains releases whose first release date falls between 1 December 2024 and 1 July 2025. From this population we uniformly sample 2000 albums and deduplicate them in the same way as the Billboard data.

To ensure a clean distinction between successful and normal albums, we remove any sampled MusicBrainz album that also appears in the Billboard 200 set above (based on the same album identifier). The remaining albums are treated as “normal” non-charting releases and are assigned the label

- `label = 0`.

6.1.3 Final sample and label balance After combining the two sources and performing all deduplication and exclusion steps, the final labeled data set contains:

- 461 successful albums (`label = 1`),
- 1623 normal albums (`label = 0`).

In total there were 2084 albums, with the positive class representing about 22% of all observations. Each album is associated with a cover image stored as a JPEG file (identified by a filename column such as `Unnamed: 0`).

This labeled sample corresponds to the “current market environment” for new releases in late 2024 and the first half of 2025 and plays the same role, for supervised learning, that the release universe in Section 5 does for unsupervised clustering.

6.2 Visual feature set and link to the clustering stage

From each cover image we extracted a set of hand-crafted features capturing color, texture, edges, composition, and the presence of people or faces. In total, each album is represented by 79 numeric features. These features largely coincide with the feature groups analyzed in the unsupervised clustering stage and therefore allow us to trace supervised results back to interpretable visual patterns.

The main feature groups are:

- **Texture (LBP).** Local Binary Pattern (LBP) descriptors summarize local texture around each pixel. We compute a 32-dimensional LBP histogram, stored as `lbp_0–lbp_31`. These features capture patterns such as smooth regions, fine-grained noise, and repetitive structure. The same LBP features underpin the texture clustering in Section 5.3.
- **Color and brightness.** We computed several statistics in HSV and RGB color spaces, including:
 - histogram bins of hue and saturation (`h_*`, `s_*`);
 - mean RGB intensities (`rgb_mean_R`, `rgb_mean_G`, `rgb_mean_B`). These summarize the overall color palette and saturation level of the cover and connect directly to the RGB and HSV clustering results in Sections 5.4–5.5.
- **Edges and structural complexity.** To quantify edge strength and visual complexity, we computed:
 - the mean magnitude of a Sobel edge detector (`sobel_mean`);
 - the fraction of pixels classified as edges (`edge_density`). These are the edge features previously clustered in Section 5.2.
- **Human presence and faces.** Using object and face detectors, we constructed variables that encode whether the cover depicts people and how prominent they are:
 - `has_person`: indicator that at least one person is detected;
 - `person_area_ratio`: ratio of the detected person area to the entire image;
 - `face_flag`, `num_faces`: presence and count of detected faces.
- **Composition and subject position.** Finally, we computed descriptors related to composition and where the main subject lies in the frame:
 - `subject_cx`, `subject_cy`: normalized subject centroid coordinates;
 - `subject_center_offset`: distance from the image center;
 - `subject_rule_of_thirds_dist`: distance to the nearest rule-of-thirds line.

Formally, the resulting feature matrix and label vector are

$$X = \text{df}[\text{feature_cols}].\text{values} \in \mathbb{R}^{2084 \times 79}, \quad y = \text{df}["\text{label}"].\text{values} \in \{0, 1\}^{2084}.$$

The unsupervised stage helps us prioritize certain subsets of these features (for example, well-separated HSV clusters or edge-complexity groups) and provides a set of interpretable style clusters that can be carried over into extended supervised models.

6.3 Train–test split and evaluation framework

To evaluate models fairly and avoid overfitting, we partitioned the data into training and test sets using an 80/20 split with stratification by label:

- $(X_{\text{train}}, y_{\text{train}})$: 1667 albums,
- $(X_{\text{test}}, y_{\text{test}})$: 417 albums.

Stratification ensures that the proportion of successful albums in the training and test sets is similar to the overall data. The full matrix X and vector y are retained for potential cross-validation.

6.3.1 Common evaluation routine To make model comparison consistent, we implemented a common evaluation routine that performs the following steps for any classifier:

1. Fit the model on $(X_{\text{train}}, y_{\text{train}})$.
2. Obtain predictions on the test set: $\hat{y}_{\text{test}}$ and, when available, the predicted probability of success $p_{\text{test}} = \Pr(y = 1 \mid X_{\text{test}})$.
3. Report the test-set classification metrics: precision, recall, and F1-score for each class, overall accuracy, and the confusion matrix.
4. Compute ROC–AUC on the test set using p_{test} .
5. Plot the ROC curve.

For some models we also ran 5-fold cross-validation over the full data set to check robustness of the ROC–AUC, although the detailed cross-validation results are omitted here for brevity.

In addition to performance metrics, we record predictions and predicted probabilities for each album. For every model we add two columns to `train_full.csv` and `test_full.csv`:

- `pred_{model}`: predicted label (0 or 1);
- `proba_{model}`: predicted probability of success.

This enables systematic error analysis and future comparison across models, including potential extensions such as mixture-of-experts architectures based on the clusters from Section 5.

6.4 Model specifications

We considered three supervised learning models: an L1-regularized logistic regression as a linear baseline, and two tree-based ensemble methods (random forest and gradient boosting) that can capture non-linear interactions between feature groups.

6.4.1 L1-regularized logistic regression As a linear baseline, we fit a logistic regression with L1 regularization. Using scikit-learn, the model is implemented as a pipeline: standardization of all features via `StandardScaler`, followed by `LogisticRegression` with:

- `penalty = "l1"`,
- `solver = "saga"`,
- regularization strength `C = 0.5`,
- `class_weight = "balanced"` to compensate for class imbalance,
- `max_iter = 5000`.

L1 regularization encourages sparsity in the coefficient vector and effectively performs feature selection. This allows us to see which individual features (or feature blocks) are most strongly associated with chart success.

6.4.2 Random forest The first non-linear model is a random forest classifier with the following hyperparameters:

- `n_estimators = 400`,
- `max_depth = None` (trees grow until leaves are pure),
- `min_samples_leaf = 2` to reduce overfitting,
- `class_weight = "balanced"`,
- `n_jobs = -1` to use all CPU cores.

Random forests can capture complex interactions between visual features and provide a measure of feature importance based on impurity reduction.

6.4.3 Gradient boosting The second tree-based method is gradient boosting, implemented via `GradientBoostingClassifier` with:

- `n_estimators = 250`,
- `learning_rate = 0.05`,
- base tree depth `max_depth = 3`,
- `subsample = 0.9` to introduce stochasticity and regularization.

Compared to random forests, gradient boosting typically produces a stronger, more focused ensemble at the cost of more sensitivity to hyperparameters.

6.5 Predictive performance

Table 1 summarizes the key test-set metrics for the three models. We focus on overall accuracy, ROC-AUC, and the precision and recall for the positive class (successful albums).

Model	Accuracy	ROC– AUC	Precision (success)	Recall (success)
Logistic Regression (L1)	0.8225	0.87	0.579	0.717
Random Forest	0.8058	0.77	0.649	0.261
Gradient Boosting	0.8297	0.79	0.733	0.359

6.5.1 Logistic regression For L1-logistic regression, the test-set classification report indicated:

- Overall accuracy: 0.8225 (343 / 417 albums correctly classified).
- ROC–AUC: approximately 0.87.
- For normal albums ($y = 0$): precision 0.91, recall 0.85, F1-score 0.88 with 325 examples.
- For successful albums ($y = 1$): precision 0.58, recall 0.72, F1-score 0.64 with 92 examples.

The confusion matrix is

$$\begin{pmatrix} 277 & 48 \\ 26 & 66 \end{pmatrix},$$

where rows correspond to true labels and columns to predicted labels. Thus the model correctly identifies 66 successful albums and misses 26, while mistakenly labeling 48 normal albums as successful. Given the relatively small number of successful albums, this represents a reasonable trade-off between false positives and false negatives.

6.5.2 Random forest For the random forest:

- Accuracy: 0.8058.
- ROC–AUC: approximately 0.77.
- For successful albums: precision 0.65, recall 0.26, F1-score 0.37.

The confusion matrix

$$\begin{pmatrix} 312 & 13 \\ 68 & 24 \end{pmatrix}$$

shows that the random forest is extremely conservative for the positive class: it correctly flags only 24 successful albums and misses 68. In other words, it almost never predicts success unless it is very confident, which yields high precision but very low recall for charting albums.

6.5.3 Gradient boosting Gradient boosting achieves the highest accuracy but only slightly improves on random forest in terms of ROC–AUC:

- Accuracy: 0.8297.
- ROC–AUC: approximately 0.79.

- For successful albums: precision 0.73, recall 0.36, F1-score 0.48.

Its confusion matrix

$$\begin{pmatrix} 313 & 12 \\ 59 & 33 \end{pmatrix}$$

indicates that the model correctly identifies 33 successful albums and misses 59. Compared to the random forest, gradient boosting is less conservative: it identifies more successful albums at the cost of some additional false positives, but its recall is still substantially lower than that of logistic regression.

6.6 Feature importance, discussion, and limitations

6.6.1 Feature importance For the L1-logistic regression, 79 features were included in the model and 66 have non-zero coefficients. Ranking features by the absolute value of their coefficients, the top positions are dominated by LBP descriptors such as `lbp_10`, `lbp_27`, `lbp_16`, `lbp_5`, and `lbp_4`. Some of these coefficients are positive, indicating that higher values are associated with an increased probability of success, while others are negative, suggesting associations with normal albums.

Because individual LBP dimensions are not easily interpretable, we interpreted them as capturing different local texture regimes: some texture patterns—for example, smoother backgrounds or certain fine-grained noise patterns—are more characteristic of successful covers, whereas other textures may be more typical of generic or cluttered designs.

For the random forest and gradient boosting models, impurity-based feature importances show a similar picture:

- Many of the same LBP features appear in the top 20 (e.g. `lbp_26`, `lbp_11`, `lbp_27`, `lbp_7`, `lbp_8`, `lbp_14`, `lbp_10`, `lbp_18`).
- Edge-related metrics (`sobel_mean`, `edge_density`) also rank highly.
- Human and composition features are consistently important: `person_area_ratio`, `has_person`, `face_flag`, and `subject_center_offset`, `subject_rule_of_thirds_dist`, `subject_cy` all appear among the top predictors.

Taken together, the three models agree that:

1. Local texture patterns (LBP features),
2. Edge strength and structural complexity, and
3. The presence, size, and placement of human figures

are the primary visual cues associated with Billboard success in this data. This is consistent with the cluster structures uncovered in Section 5, where distinct texture, edge, and color regimes emerged.

6.6.2 Discussion and limitations From a purely predictive standpoint, the L1-regularized logistic regression performs best: it achieves the highest ROC–AUC and substantially higher recall for successful albums than the tree-based models. This is important for the intended use case: if the model is used as a screening tool to identify potentially successful releases, missing many true hits would be costly, whereas a moderate number of false positives is acceptable. The tree-based models, particularly random forest, behave more conservatively. They identify fewer hits, but the ones they do identify have higher precision (especially for gradient boosting). Such models might be useful in scenarios where we want a small, highly confident shortlist of covers with a strong visual signal.

Overall, this supervised learning design links the exploratory clustering of color, edge, and texture features to a concrete prediction task on recent releases. Across all models, local texture descriptors, edge-based measures, and human/composition features emerge as the most important predictors. These findings suggest that there are systematic, quantifiable visual patterns in successful album covers and that even relatively simple models can exploit them to anticipate commercial outcomes.

7 Assumptions and Limitations

There are several limitations and assumptions in this study that should be noted. First, the definition of “success” is narrow: an album is considered successful if it appears on the Billboard 200 between January and July 2025. This excludes albums that may become successful later or in other markets. Additionally, the negative class is formed by random sampling of MusicBrainz albums not seen on the Billboard chart during the same period; some of these may be niche but high-quality releases that succeed on dimensions not captured by the Billboard chart.

Another key assumption is that album cover art contains signals related to success. This study is unique in that most literature in this field focuses on audio features, and the research that inspired this study draws from other industries. Therefore, we assume that visual features of album covers contain some predictive information about chart success, even if not causal. This assumption comes with limitations when considering confounding factors, such as artist popularity, label-backed marketing budgets, and social media virality—factors that the model cannot control for and which may influence success independently of the visual features.

Further, the models rely on hand-crafted visual features extracted using sci-kit image, which focus on color, texture, and edges. We assume these features sufficiently capture the visual characteristics that matter for success. However, using pre-trained convolutional neural networks (CNNs) could potentially capture richer, deeper embeddings directly from raw pixels, improving performance and potentially integrating clustering and prediction more effectively. The choice to use hand-crafted features was made due to the need for interpretability and lower computational cost, as deriving deep embeddings for unlabeled album cover data was not feasible in this study’s context.

Additionally, the study assumes temporal stability—specifically, that visual trends within the sampling window do not significantly impact the results. This assumption is reasonable given the relatively short time frame, but there remains a risk that visual feature importance and its relationship with success may change over time, causing the model’s performance to degrade. Non-stationarity in the data may not have been fully addressed, potentially impacting the results.

Lastly, there is a selection bias in the sample, as only album covers that make it to market are included. These covers have likely gone through multiple drafts before being released, and there is

no comparison to “bad” album covers in the dataset. Therefore, the study lacks data on unsuccessful album covers, which limits the ability to compare successful designs to unsuccessful ones.

Appendix A: Data Dictionary

Variable	Variable Name	Type	Allowed values	Description
Album Title	title	String	Any string	The name of the album
Country	country	String	Any valid two letter country code	The country where this release of the album was published
Release Date	date	String	String in the format of YYYY-MM-DD or YYYY	The primary release date of this release of the album
Track Count	track_count	Integer	Any int ≥ 1	The number of songs on the album
Record Label	label_names	String	Semi colon separated names of the record labels	The record labels that published the album
Chart Date	chart_date	String	String in the format of YYYY-MM-DD	The week which the chart data is for
Artist	artist	String	Any string	The artist who created the album
Peak Position	peakPos	Integer	Any int 1-200	The peak position of the album on the Billboard 200
Last Position	lastPos	Integer	Any int 1-200	The position of the album last on week on the Billboard 200
Weeks	weeks	Integer	Any int ≥ 1	The number of weeks the album has been present on the Billboard 200
Is New	isNew	Boolean	TRUE or FALSE	Whether the album is new to the Billboard 200 or not
Cover Art URL	cover_url	String	A valid link	A link to a hosted URL of the album's cover art

Appendix B: Code

```
#### Data Collection Code ####
# Query the MusicBrainz API for all albums in a date range
def fetch_unique_album_releases_by_date_range(start_date, end_date,
                                              primary_type="Album",
                                              page_size=100,
                                              pause_sec=1.0,
                                              max_pages=None):

    """
    Fetch releases whose *release date* is within [start_date, end_date]
    and whose primary type is `primary_type`, by iterating over
    paginated MusicBrainz results.

    We then DEDUPLICATE by `release-group.id`, so that each album
    (release-group) appears at most once.

    Returns: a list of release dicts, one per unique album.
    """
    all_releases = []
    seen_release_group_ids = set()

    offset = 0
    page = 0

    while True:
        query = f'date:[{start_date} TO {end_date}] AND primarytype:{primary_type}'
        params = {
            "fmt": "json",
            "query": query,
            "limit": page_size,
            "offset": offset,
        }

        print(f"Requesting page {page} (offset={offset}) ...")
        resp = requests.get(MB_BASE_URL_REL, params=params, headers=HEADERS)
        resp.raise_for_status()
        data = resp.json()

        releases = data.get("releases", [])
        total_count = data.get("count", 0)
        print(f" Got {len(releases)} releases on this page; total_count={total_count}")

        if not releases:
            break

    # Deduplicate by release-group ID (album-level)
```

```

for rel in releases:
    rg = rel.get("release-group") or {}
    rgid = rg.get("id")
    if not rgid:
        continue

    if rgid in seen_release_group_ids:
        continue

    seen_release_group_ids.add(rgid)
    all_releases.append(rel)

offset += page_size
page += 1

if offset >= total_count:
    break

if max_pages is not None and page >= max_pages:
    print("Reached max_pages limit, stopping early.")
    break

sleep(pause_sec)

print(f"Total unique albums collected (by release-group id): {len(all_releases)}")
return all_releases
# Flatten MusicBrainz data into pandas Dataframe
def releases_to_dataframe(releases):
    """
    Convert a list of release dicts into a tidy pandas DataFrame.
    We flatten all relevant MusicBrainz fields and also add a
    Cover Art Archive front-cover URL for each release.
    """
    rows = []

    for rel in releases:
        # --- artist-credit (flatten names) ---
        artist_credits = rel.get("artist-credit", [])
        artist_names = []
        for ac in artist_credits:
            if isinstance(ac, dict):
                if "artist" in ac and isinstance(ac["artist"], dict):
                    artist_names.append(ac["artist"].get("name"))
                elif "name" in ac:
                    artist_names.append(ac.get("name"))
        artist_credit_str = " & ".join(a for a in artist_names if a)

```

```

# --- label-info (flatten labels and catalog numbers) ---
label_names = []
label_ids = []
catalog_numbers = []
for li in rel.get("label-info", []):
    if not isinstance(li, dict):
        continue
    lab = li.get("label") or {}
    if "name" in lab:
        label_names.append(lab["name"])
    if "id" in lab:
        label_ids.append(lab["id"])
    if "catalog-number" in li:
        catalog_numbers.append(li["catalog-number"])

# --- media (flatten basic info) ---
media_list = rel.get("media", [])
media_count = len(media_list)
media_formats = set()
total_disc_count = 0
for m in media_list:
    if not isinstance(m, dict):
        continue
    fmt = m.get("format")
    if fmt:
        media_formats.add(fmt)
    total_disc_count += m.get("disc-count", 0)

# --- release-events (flatten dates and area names) ---
event_dates = []
event_areas = []
for ev in rel.get("release-events", []):
    if not isinstance(ev, dict):
        continue
    if "date" in ev:
        event_dates.append(ev["date"])
    area = ev.get("area") or {}
    if "name" in area:
        event_areas.append(area["name"])

# --- release-group (flatten basic info) ---
rg = rel.get("release-group", {}) or {}
rg_id = rg.get("id")
rg_title = rg.get("title")
rg_primary_type = rg.get("primary-type")

# --- tags (flatten tag names) ---

```

```

tags_list = rel.get("tags", []) or []
tag_names = []
for t in tags_list:
    if isinstance(t, dict) and "name" in t:
        tag_names.append(t["name"])

# --- text-representation (language, script) ---
text_rep = rel.get("text-representation", {}) or {}
lang = text_rep.get("language")
script = text_rep.get("script")

release_id = rel.get("id")

rows.append({
    # core identifiers
    "release_id": release_id,
    "release_group_id": rg_id,
    "release_group_title": rg_title,
    "release_group_primary_type": rg_primary_type,

    # simple scalar fields (direct from MusicBrainz)
    "title": rel.get("title"),
    "artist_credit": artist_credit_str,
    "artist_credit_id": rel.get("artist-credit-id"),
    "asin": rel.get("asin"),
    "barcode": rel.get("barcode"),
    "count": rel.get("count"),
    "country": rel.get("country"),
    "date": rel.get("date"),
    "disambiguation": rel.get("disambiguation"),
    "score": rel.get("score"),
    "status": rel.get("status"),
    "status_id": rel.get("status-id"),
    "track_count": rel.get("track-count"),
    "packaging": rel.get("packaging"),
    "packaging_id": rel.get("packaging-id"),

    # flattened label-info
    "label_names": "; ".join(label_names),
    "label_ids": "; ".join(label_ids),
    "label_catalog_numbers": "; ".join(catalog_numbers),

    # flattened media info
    "media_count": media_count,
    "media_formats": "; ".join(sorted(media_formats)),
    "media_total_disc_count": total_disc_count,

```

```

        # flattened release-events
        "release_event_dates": "; ".join(event_dates),
        "release_event_areas": "; ".join(event_areas),

        # flattened tags
        "tags": "; ".join(tag_names),

        # text representation
        "language": lang,
        "script": script,

    })

    df = pd.DataFrame(rows)
    return df
# Query MusicBrainz for albums released in a time range and write output to CSV
releases = fetch_unique_album_releases_by_date_range(
    START_DATE, END_DATE, PRIMARY_TYPE,
    page_size=100, pause_sec=1.0
)
releases_df = releases_to_dataframe(releases)

n_rows = len(releases_df)
n_unique_rg = releases_df["release_group_id"].nunique()
print(f"DataFrame rows: {n_rows}")
print(f"Unique release_group_id: {n_unique_rg}")

if n_rows != n_unique_rg:
    print("WARNING: rows and unique release_group_id do not match.")
else:
    print("OK: one row per album (release-group).")

csv_name = f"musicbrainz_albums_{START_DATE}_to_{END_DATE}.csv"
csv_path = os.path.join(SAVE_DIR, csv_name)
releases_df.to_csv(csv_path, index=False)

print(f"Saved album metadata to: {csv_path}")
releases_df.head()

## Query Billboard 200 Data
import re
import requests

```

```

import pandas as pd
import datetime as dt
import billboard
from tqdm import tqdm

CHART_NAME = "billboard-200"
START_DATE = "2024-07-01"
END_DATE = "2024-09-15"

SAVE_DIR = "/content/drive/MyDrive/Music Capstone/Data Collection"
COVER_DIR = os.path.join(SAVE_DIR, "covers")
os.makedirs(SAVE_DIR, exist_ok=True)
os.makedirs(COVER_DIR, exist_ok=True)

OUTPUT_CSV = os.path.join(
    SAVE_DIR, f"billboard_{CHART_NAME}_{START_DATE}_to_{END_DATE}.csv"
)

def _to_date(s: str) -> dt.date:
    return dt.datetime.strptime(s, "%Y-%m-%d").date()

def _sanitize_filename(name: str) -> str:
    name = re.sub(r"[\\/:*?\"<>|]", "_", name)
    name = re.sub(r"\s+", " ", name).strip()
    return name[:160]

def _pick_ext_from_url(url: str) -> str:
    if not url:
        return ".jpg"
    m = re.search(r"\.(jpg|jpeg|png|webp|gif)(?:\?|$)", url, flags=re.I)
    return f".{m.group(1).lower()}" if m else ".jpg"

def _download_image(url: str, save_path: str, timeout=15) -> bool:
    try:
        r = requests.get(url, stream=True, timeout=timeout)
        if r.status_code == 200:
            with open(save_path, "wb") as f:
                for chunk in r.iter_content(1024 * 16):
                    if chunk:
                        f.write(chunk)
            return True
    
```

```

except Exception as e:
    pass
return False

def fetch_weeks_in_range(chart_name: str, start_date: str, end_date: str) -> pd.DataFrame:
    start_d = _to_date(start_date)
    end_d = _to_date(end_date)
    assert start_d <= end_d, "START_DATE <= END_DATE"

    chart = billboard.ChartData(chart_name, date=end_date)
    while chart.date and _to_date(chart.date) > end_d and chart.previousDate:
        chart = billboard.ChartData(chart_name, date=chart.previousDate)

    charts = []
    seen_dates = set()

    if chart.date:
        charts.append(chart)
        seen_dates.add(chart.date)

    while chart and chart.previousDate:
        prev = billboard.ChartData(chart_name, date=chart.previousDate)
        if not prev.date:
            break
        if prev.date not in seen_dates:
            charts.append(prev)
            seen_dates.add(prev.date)
        chart = prev
        if _to_date(chart.date) < start_d:
            break

    if charts:
        min_date = min(_to_date(c.date) for c in charts if c.date)
        while min_date > start_d:
            probe = (min_date - dt.timedelta(days=7)).isoformat()
            probe_chart = billboard.ChartData(chart_name, date=probe)
            if not probe_chart.date:
                break
            if probe_chart.date not in seen_dates:
                charts.append(probe_chart)
                seen_dates.add(probe_chart.date)
                min_date = min(min_date, _to_date(probe_chart.date))
            else:
                break

    charts = [c for c in charts if c.date and start_d <= _to_date(c.date) <= end_d]

```

```

rows = []
for ch in sorted(charts, key=lambda c: c.date):
    cdate = ch.date
    for e in ch:
        rows.append({
            "chart": chart_name,
            "chart_date": cdate,
            "rank": e.rank,
            "title": e.title,
            "artist": e.artist,
            "peakPos": getattr(e, "peakPos", None),
            "lastPos": getattr(e, "lastPos", None),
            "weeks": getattr(e, "weeks", None),
            "isNew": getattr(e, "isNew", None),
            "coverurl": getattr(e, "image", None),
        })

df = pd.DataFrame(rows)
if not df.empty:
    df = df.sort_values(["chart_date", "rank"], ascending=[True, True]).reset_index(drop=True)
return df

df = fetch_weeks_in_range(CHART_NAME, START_DATE, END_DATE)

if df.empty:
    print("df empty")
else:
    uniq_weeks = sorted(df["chart_date"].unique())
    print(uniq_weeks)

df = download_covers(df, COVER_DIR)

df.to_csv(OUTPUT_CSV, index=False, encoding="utf-8-sig")
print(f"\n albuminfo saved: \n{OUTPUT_CSV}")
print(f" covers saved: \n{COVER_DIR}")

print(df.head(10))

###Feature Extraction and EDA###

# 3. Enhanced Feature Extraction
from scipy.stats import skew, kurtosis

def extract_features(img):

```

```

# Grayscale conversion (for texture and edge analysis)
img_gray = color.rgb2gray(img)

# HSV color histograms (8 bins each)
hsv = color.rgb2hsv(img)
hist_h = np.histogram(hsv[...,0], bins=8, range=(0,1), density=True)[0]
hist_s = np.histogram(hsv[...,1], bins=8, range=(0,1), density=True)[0]
hist_v = np.histogram(hsv[...,2], bins=8, range=(0,1), density=True)[0]
color_feat = np.concatenate([hist_h, hist_s, hist_v])

# RGB channel statistics
# Compute mean, std, skewness, and kurtosis for each color channel
r, g, b = img[...,0], img[...,1], img[...,2]
rgb_means = [r.mean(), g.mean(), b.mean()]
rgb_stds = [r.std(), g.std(), b.std()]
rgb_skews = [skew(r.flatten()), skew(g.flatten()), skew(b.flatten())]
rgb_kurts = [kurtosis(r.flatten()), kurtosis(g.flatten()), kurtosis(b.flatten())]
rgb_stats = np.concatenate([rgb_means, rgb_stds, rgb_skews, rgb_kurts])

# Texture (Local Binary Pattern)
lbp = feature.local_binary_pattern(img_gray, P=8, R=1)
lbp_hist = np.histogram(lbp, bins=32, range=(0,256), density=True)[0]

# Edge features (Canny + Sobel)
edges = feature.canny(img_gray, sigma=1.5)
sobel_mean = filters.sobel(img_gray).mean()
edge_density = edges.mean()

# Combine all features
return np.concatenate([color_feat, rgb_stats, lbp_hist, [sobel_mean, edge_density]])

# 4. Extract features for all samples (progress bar updates every 5%)
import math

feature_list, names = [], []
total = len(sample_imgs)
step = max(1, math.ceil(total * 0.05)) # update every 5%

pbar = tqdm(total=total, desc="Extracting features (5% steps)", ncols=100)
count = 0

for path in sample_imgs:
    img = load_and_preprocess(path)
    if img is None:
        continue

    feats = extract_features(img)

```

```

feature_list.append(feats)
names.append(os.path.basename(path))

count += 1
if (count % step == 0) or (count == total):
    pbar.update(step)

pbar.close()

# == Column names (including new RGB statistics)
columns = (
    [f"h_{i}" for i in range(8)] +
    [f"s_{i}" for i in range(8)] +
    [f"v_{i}" for i in range(8)] +
    [f"rgb_mean_{c}" for c in ["R", "G", "B"]] +
    [f"rgb_std_{c}" for c in ["R", "G", "B"]] +
    [f"rgb_skew_{c}" for c in ["R", "G", "B"]] +
    [f"rgb_kurt_{c}" for c in ["R", "G", "B"]] +
    [f"lbp_{i}" for i in range(32)] +
    ["sobel_mean", "edge_density"]
)

df_features = pd.DataFrame(feature_list, columns=columns, index=names)
df_features.to_csv(OUTPUT_CSV)
print(f" Saved enhanced feature CSV → {OUTPUT_CSV}")
df_features.head()

#### Feature Engineering Pipeline for Edge, Texture, RGB, and HSV Features
### This section outlines the feature engineering pipeline, using HSV features
### as an example. Note that slight variations exist across these pipelines due
### to the inherent differences among feature types. For instance, PCA was not
### applied to the Edge features, since this category contains only two variables,
### making dimensionality reduction unnecessary.####
# 1. Imports & Parameters

import os
import random
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from tqdm import tqdm

```

```

from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
from sklearn.metrics import Silhouette_score
from matplotlib.offsetbox import OffsetImage, AnnotationBbox

# Paths
COVER_DIR = "/content/drive/MyDrive/Music Capstone/Data Collection/covers"
CSV_PATH = "/content/drive/MyDrive/Music Capstone/Feature Engineering/Demo_315_1105.csv"

random.seed(42)
np.random.seed(42)

# 2. Load CSV and standardize HSV features

# Load CSV; index_col=0 is the image ID / full file name
df = pd.read_csv(CSV_PATH, index_col=0)
print("Raw feature table shape:", df.shape)

# Select HSV columns: columns B to Y (0-based index 0:24)
hsv_cols = df.columns[0:24]
selected_cols = list(hsv_cols)

# If you want RGB only, swap to:
# rgb_cols = df.columns[25:37]
# selected_cols = list(rgb_cols)

# Standardize selected features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(df[selected_cols])
print("Normalized feature matrix shape:", X_scaled.shape)

# 3. Inspect explained variance for all principal components

# Fit PCA with all possible components
pca_full = PCA() # by default n_components = min(n_samples, n_features)
pca_full.fit(X_scaled)

```

```

# Explained variance ratio for each PC
explained_var_ratio = pca_full.explained_variance_ratio_

print("Number of PCs:", len(explained_var_ratio))
print("Explained variance ratio (first 10 PCs):")
for i, v in enumerate(explained_var_ratio[:10], start=1):
    print(f"PC{i}: {v:.4f}")

# Scree plot: variance explained by each PC
plt.figure(figsize=(6, 4))
plt.plot(range(1, len(explained_var_ratio) + 1),
         explained_var_ratio,
         marker="o")
plt.xlabel("Principal Component index")
plt.ylabel("Explained variance ratio")
plt.title("Scree plot of PCA components")
plt.tight_layout()
plt.show()

# Cumulative explained variance
cum_explained = np.cumsum(explained_var_ratio)

plt.figure(figsize=(6, 4))
plt.plot(range(1, len(cum_explained) + 1),
         cum_explained,
         marker="o")
plt.xlabel("Principal Component index")
plt.ylabel("Cumulative explained variance ratio")
plt.title("Cumulative explained variance")
plt.axhline(y=0.8, linestyle="--") # for example, 80% threshold
plt.tight_layout()
plt.show()

# 4a. Compare PCA dimensions with k-means via Silhouette

def evaluate_pca_kmeans(X_scaled, n_components_list, k_range, random_state=42):
    """
    For each PCA dimension in n_components_list:
    - fit PCA
    - run k-means for all K in k_range
    - compute Silhouette scores
    - keep the best K and clustering result
    """

```

```

results = {}

for n in n_components_list:
    # PCA transform
    pca = PCA(n_components=n, random_state=random_state)
    X_pca = pca.fit_transform(X_scaled)

    scores = []
    models = []
    labels_list = []
    k_values = list(k_range)

    for k in k_values:
        km = KMeans(n_clusters=k, n_init=10, random_state=random_state)
        labels = km.fit_predict(X_pca)
        score = Silhouette_score(X_pca, labels)

        scores.append(score)
        models.append(km)
        labels_list.append(labels)

    best_idx = int(np.argmax(scores))

    results[n] = {
        "pca": pca,
        "X_pca": X_pca,
        "k_values": k_values,
        "scores": scores,
        "best_k": k_values[best_idx],
        "best_score": scores[best_idx],
        "best_model": models[best_idx],
        "best_labels": labels_list[best_idx],
    }

# Plot Silhouette scores for each PCA dimension
plt.figure(figsize=(7, 5))
for n in n_components_list:
    plt.plot(results[n]["k_values"],
             results[n]["scores"],
             marker="o",
             label=f"{n} PCs")
plt.xlabel("Number of clusters K")
plt.ylabel("Silhouette score")
plt.title("K-means performance under different PCA dimensions")
plt.legend()

```

```

plt.tight_layout()
plt.show()

# Print best K summary
for n in n_components_list:
    print(
        f"PCA with {n} components -> "
        f"best K = {results[n]['best_k']}, "
        f"Silhouette = {results[n]['best_score']:.3f}"
    )

return results

# run for the PCA options you care about
n_components_list = [2, 5, 8]          # you can add more, e.g. 10, 12, ...
k_range = range(2, 11)                # try K = 2,...,10

pca_kmeans_results = evaluate_pca_kmeans(
    X_scaled,
    n_components_list=n_components_list,
    k_range=k_range,
    random_state=42,
)

# 4b. Visualize best k-means clustering on 2D PCA (n_components=2)

# Get PCA(2) info and best clustering from previous cell
pca2_info = pca_kmeans_results[2]
X_pca2 = pca2_info["X_pca"]
labels2 = pca2_info["best_labels"]
best_k2 = pca2_info["best_k"]

# Print explained variance for 2 PCs
explained2 = pca2_info["pca"].explained_variance_ratio_.sum()
print(f"Explained variance (2 PCs): {explained2:.4f}")
print(f"Best K for 2 PCs: {best_k2}")

```

```

# 2D scatter of the clusters on PC1 vs PC2
plt.figure(figsize=(6, 5))
sc = plt.scatter(
    X_pca2[:, 0],
    X_pca2[:, 1],
    c=labels2,
    s=10,
    alpha=0.7,
)
plt.title(f"PCA 2D clusters (PC1 vs PC2), K = {best_k2}")
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.colorbar(sc, label="Cluster")
plt.tight_layout()
plt.show()

# 5. Feature loadings and contributions for each PCA option

def compute_feat_table(pca_obj, feature_names):
    """Return loadings + normalized contributions for one PCA object."""
    comps = pca_obj.components_          # (n_pc, n_feat)
    n_pc = comps.shape[0]

    # loadings: coefficients of each feature in each PC
    loadings = pd.DataFrame(
        comps.T,
        index=feature_names,
        columns=[f"PC{i+1}" for i in range(n_pc)]
    )

    # contributions: squared loadings, normalized per PC
    contrib = loadings**2
    contrib = contrib.div(contrib.sum(axis=0), axis=1)
    contrib.columns = [f"{c}_contrib" for c in loadings.columns]

    table = pd.concat([loadings, contrib], axis=1)
    return table

feat_pc_tables = {}    # {n_components: DataFrame}
top_k = 10             # top features to display

```

```

for n in n_components_list:    # [2, 5, 8]
    info = pca_kmeans_results[n]
    pca_n = info["pca"]

    print(f"\n=== PCA with {n} components ===")
    table = compute_feat_table(pca_n, selected_cols)
    display(table.head())
    feat_pc_tables[n] = table

    # show top-k features for each PC of this PCA
    for i in range(pca_n.n_components_):
        pc_name = f"PC{i+1}"
        contrib_col = f"{pc_name}_contrib"
        print(f"Top {top_k} features for {pc_name}:")
        display(
            table.sort_values(contrib_col, ascending=False).head(top_k)
        )
# A. Internal validation metrics for k-means on PCA(2)

from sklearn.cluster import KMeans
from sklearn.metrics import (
    Silhouette_score,
    calinski_harabasz_score,
    davies_bouldin_score,
)
import numpy as np
import matplotlib.pyplot as plt

K_range = range(2, 12)    # K = 2,...,11

sil_scores = []
ch_scores = []
db_scores = []

for k in K_range:
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels_k = km.fit_predict(X_pca2)    # use 2D PCA embedding

    sil_scores.append(Silhouette_score(X_pca2, labels_k))
    ch_scores.append(calinski_harabasz_score(X_pca2, labels_k))

```

```

db_scores.append(davies_bouldin_score(X_pca2, labels_k))

# --- Silhouette ---
plt.figure(figsize=(5, 4))
plt.plot(list(K_range), sil_scores, marker="o")
plt.title("Silhouette score vs. K (PCA 2D)")
plt.xlabel("K")
plt.ylabel("Silhouette score")
plt.grid(True)
plt.tight_layout()
plt.show()

# --- Calinski-Harabasz ---
plt.figure(figsize=(5, 4))
plt.plot(list(K_range), ch_scores, marker="o")
plt.title("Calinski-Harabasz index vs. K (PCA 2D)")
plt.xlabel("K")
plt.ylabel("CH index (higher is better)")
plt.grid(True)
plt.tight_layout()
plt.show()

# --- Davies-Bouldin ---
plt.figure(figsize=(5, 4))
plt.plot(list(K_range), db_scores, marker="o")
plt.title("Davies-Bouldin index vs. K (PCA 2D)")
plt.xlabel("K")
plt.ylabel("DB index (lower is better)")
plt.grid(True)
plt.tight_layout()
plt.show()

print("Silhouette:", dict(zip(K_range, np.round(sil_scores, 3))))
print("Calinski-Harabasz:", dict(zip(K_range, np.round(ch_scores, 1))))
print("Davies-Bouldin:", dict(zip(K_range, np.round(db_scores, 3))))
# B. Clustering stability check via Adjusted Rand Index (PCA 2D, K = best_k2)

from sklearn.metrics import adjusted_rand_score

K_stab = best_k2          # best K for PCA(2) from earlier step
n_runs = 8                # number of random initializations

```

```

base_km = KMeans(n_clusters=K_stab, random_state=0, n_init=10)
base_labels = base_km.fit_predict(X_pca2)

ari_scores = []

for seed in range(1, n_runs + 1):
    km = KMeans(n_clusters=K_stab, random_state=seed, n_init=10)
    labels_seed = km.fit_predict(X_pca2)
    ari = adjusted_rand_score(base_labels, labels_seed)
    ari_scores.append(ari)

print(f"Adjusted Rand Index vs. seed (K={K_stab}, PCA 2D):")
for i, ari in enumerate(ari_scores, start=1):
    print(f"  run {i}: ARI = {ari:.3f}")

print(f"\nMean ARI: {np.mean(ari_scores):.3f}, std: {np.std(ari_scores):.3f}")

```